

Operating System Checkpoint – Task Management Simulation

Name: Sylvestre IBOMBO GAKOSSO

Date: 11/05/2026

Course / Class: Introduction to Operating Systems

Introduction

In this assignment, we simulate several essential mechanisms of an operating system: CPU scheduling, process synchronization, memory management via paging, and disk access optimization. Each section contains a simulation, calculations, and an analysis.

Part 1: Process Scheduling

Given Processes

Process	Arrival Time	Burst Time
P1	0	5
P2	1	3
P3	2	1

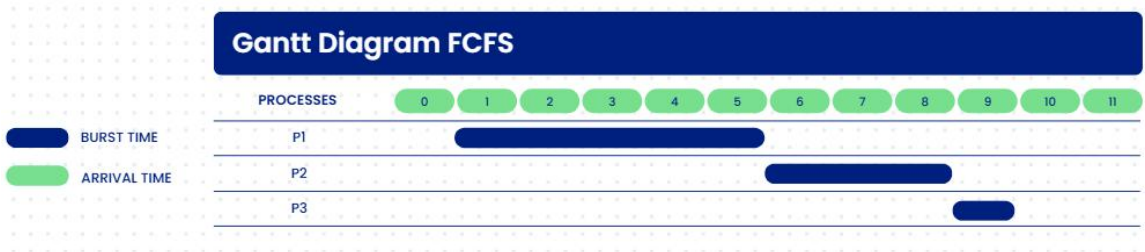
1. FCFS Scheduling

Execution Order: They executes by their arrival order

P1 → P2 → P3 (based on arrival time)

- P1 : burst = 5 --> 0 à 5
- P2 : burst = 3 --> 5 à 8
- P3 : burst = 1 --> 8 à 9

Gantt Chart (FCFS)



Waiting Time Calculation

· P1 : start = 0, arrival = 0
 WT = 0 - 0 = 0

· P2 : start = 5, arrival = 1
 WT = 5 - 1 = 4

· P3 : start = 8, arrival = 2
 WT = 8 - 2 = 6

Average Waiting Time (FCFS)

Average WT = (WT1 + WT2 + WT3) / 3

Average WT = (0 + 4 + 6) / 3

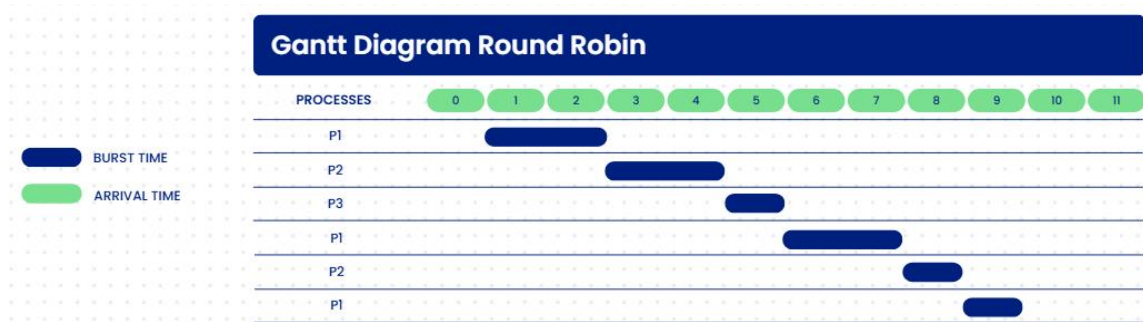
Average WT = 3,33

2. Round Robin Scheduling (Quantum = 2)

Execution Order

- P1 : 0 → 2 (remaining 3)
- P2 : 2 → 4 (remaining 1)
- P3 : 4 → 5 (remaining)
- P1 : 5 → 7 (remaining 1)
- P2 : 7 → 8 (finishes)
- P1 : 8 → 9 (finishes)

Gantt Chart (Round Robin)



Waiting Time Calculation

$$\text{Waiting Time} = \text{Finish Time} - \text{Arrival Time} - \text{Burst Time}$$

- P1 ends at 9
- P2 ends at 8
- P3 ends at 5

$$\text{Waiting Time P1} = 9 - 0 - 5 = 4$$

$$\text{Waiting Time P2} = 8 - 1 - 3 = 4$$

$$\text{Waiting Time P3} = 5 - 2 - 1 = 2$$

Average Waiting Time (RR)

$$\text{Average WT} = (\text{WT1} + \text{WT2} + \text{WT3}) / 3$$

$$\text{Average WT} = (4 + 4 + 2) / 3$$

$$\text{Average WT} = 3,33$$

3. Comparison (FCFS vs Round Robin)

Which algorithm is more responsive and why?

Round Robin is more responsive because it ensures that no process waits too long before getting CPU time. FCFS can delay short processes behind long ones, increasing response time and reducing fairness.

However, in this case both algorithms give the same average waiting time, but RR remains better in interactive systems due to better time-sharing.

Part 2: Process Synchronization

Scenario

Two processes P1 and P2 access a shared variable counter. Each process increments counter 100 times.

1. What could go wrong without synchronization?

Without synchronization, both processes may access and update counter simultaneously. This causes a **race condition**, where increments are lost due to concurrent overwriting. The final value will likely be less than 200 instead of the expected result.

2. Proposed Synchronization Mechanism

I chose the mutex.

Reason:

A mutex is appropriate because:

- Only two processes are involved

- We need strict mutual exclusion

- It is simple to implement and efficient for this case

3. Pseudocode with Mutual Exclusion

Shared variable:

```
counter = 0
```

Process P1:

```
for i = 1 to 100:
```

```
  lock(mutex)
```

```
  counter = counter + 1
```

```
  unlock(mutex)
```

Process P2:

```
for i = 1 to 100:
lock(mutex)
counter = counter + 1
unlock(mutex)
```

Part 3: Memory Management

Scenario

Process needs 5 pages. System has only 3 page frames.

Page reference string: 1, 2, 3, 2, 4, 1, 5

1. FIFO Page Replacement

FIFO replaces the oldest loaded page regardless of usage. This often causes inefficiency because frequently used pages may be removed.

Simulation Table:

Reference	Frame 1	Frame 2	Frame 3	Page Fault (Yes/No)
1	1			Yes
2	1	2		Yes
3	1	2	3	Yes
2	1	2	3	No
4	4	2	3	Yes
1	4	1	3	Yes
5	4	1	5	Yes

Total Page Faults (FIFO) = 6 (There are 1, 2, 3, 4, 1, 5)

2. LRU Page Replacement

LRU replaces the least recently used page, improving performance by keeping active pages in memory longer.

Simulation Table:

Reference	Frame 1	Frame 2	Frame 3	Page Fault (Yes/No)
1	1			Yes
2	1	2		Yes
3	1	2	3	Yes
2	1	2	3	No
4	1	2	4	Yes
1	1	2	4	No
5	1	2	5	Yes

Total Page Faults (LRU) = 5 (There are 1,2,3,4,5)

3. Comparison (FIFO vs LRU)

LRU is better because it reduces page faults by exploiting temporal locality. Programs tend to reuse recently accessed data, so LRU aligns better with real execution patterns.

FIFO is simpler but can perform poorly because it ignores actual usage behavior.

✓ Conclusion: LRU is more efficient and closer to real OS memory management strategies.

Part 4: Disk Scheduling

Scenario

Initial disk head position: 53

Pending requests: 98, 183, 37, 122, 14, 124, 65, 67

1. FCFS Disk Scheduling

Order of Service:

53 → → → → → → → →

Head Movement Calculation:

Move	From	To	Distance (to - from)
1	53	98	45
2	98	183	85
3	183	37	146
4	37	122	85
5	122	14	108
6	14	124	110
7	124	65	59
8	65	67	2

Total Head Movement (FCFS) = 640

2. SSTF Disk Scheduling

Pending requests: 98, 183, 37, 122, 14, 124, 65, 67

Order of Service:

53 → → → → → → → →

Head Movement Calculation:

Move	From	To	Distance (to - from)
1	53	65	12
2	65	67	2
3	67	37	30
4	37	14	23
5	14	98	84
6	98	122	24
7	122	124	2
8	124	183	59

Total Head Movement (SSTF) = 236

3. Conclusion (FCFS vs SSTF)

⚠ Conclue quel algorithme est le plus efficace et pourquoi.

Both are not optimal. SSTF is clearly more efficient in terms of head movement. However, it may cause **starvation** for far requests if closer requests keep arriving.

✓ Conclusion: SSTF is more efficient but not always fair.

Final Conclusion

This assignment demonstrates how operating system algorithms strongly impact performance:

CPU scheduling: Round Robin improves responsiveness and fairness compared to FCFS

Synchronization: Mutex prevents race conditions and ensures data consistency

Memory management: LRU is more efficient than FIFO due to better usage prediction

Disk scheduling: SSTF reduces head movement but can introduce starvation risks

Overall, OS design is a trade-off between performance, fairness, and complexity.